

# ❑ Connexions aux Bases de Données en Python

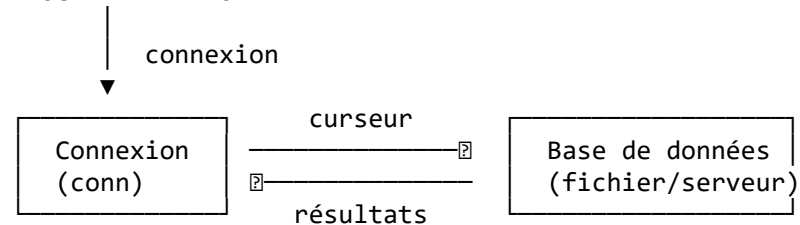
Le support est une propriété de **Bechir Bejaoui**

---

## 1 ❑ Introduction —Python et les bases de données

Une application professionnelle a presque toujours besoin de **stocker et relire des données de façon persistante**. Les fichiers texte ou CSV ont leurs limites dès qu'on a besoin de filtrer, trier, mettre à jour ou gérer plusieurs utilisateurs simultanément. C'est là qu'interviennent les **bases de données relationnelles**.

PROGRAMME PYTHON



- **connexion** (conn) : le canal de communication entre Python et la base —à ouvrir et fermer proprement
- **curseur** (cursor) : l'objet qui exécute les requêtes SQL et récupère les résultats
- **transaction** : groupe d'opérations validées ensemble (commit) ou annulées ensemble (rollback)

**Les 3 bases de ce cours :**

| Base              | Module Python          | Installation | Usage typique               |
|-------------------|------------------------|--------------|-----------------------------|
| <b>SQLite</b>     | sqlite3 (intégré)      | Aucune ❑     | Tests, apps locales, mobile |
| <b>MySQL</b>      | mysql-connector-python | pip install  | Web, e-commerce             |
| <b>PostgreSQL</b> | psycopg2               | pip install  | Systèmes critiques, finance |

---

## 2 ❑ SQLite —démarrer sans rien installer

SQLite est parfait pour apprendre : pas de serveur, pas d'installation, intégré à Python. La base est un simple fichier `.db`.

```

import sqlite3

# Connexion — crée le fichier s'il n'existe pas
conn = sqlite3.connect("boutique.db")

# Le curseur exécute les requêtes SQL
cursor = conn.cursor()

# Créer une table
cursor.execute("""
    CREATE TABLE IF NOT EXISTS produit (
        id     INTEGER PRIMARY KEY AUTOINCREMENT,
        nom    TEXT      NOT NULL,
        prix   REAL      NOT NULL
    )
""")

# Insérer un enregistrement — toujours utiliser ? pour les valeurs
cursor.execute(
    "INSERT INTO produit (nom, prix) VALUES (?, ?)",
    ("Clavier", 99.90)
)

# Valider la transaction — sans ça, l'insertion n'est PAS sauvegardée
conn.commit()

# Lire les données
cursor.execute("SELECT * FROM produit")
for ligne in cursor.fetchall():
    print(ligne)    # (1, 'Clavier', 99.9)

# Toujours fermer
cursor.close()
conn.close()

(1, 'Clavier', 99.9)

```

- **sqlite3.connect("boutique.db")** : crée le fichier boutique.db si inexistant — ":memory:" pour une base temporaire en RAM
- **?** comme placeholder : les valeurs sont passées dans un tuple séparé — Python les insère de façon sécurisée
- **conn.commit()** : **obligatoire** après toute modification — sans ça, les changements sont perdus à la fermeture
- **CREATE TABLE IF NOT EXISTS** : ne lève pas d'erreur si la table existe déjà — pratique à relancer

### 3 □ Le pattern with —fermeture automatique

La façon recommandée : utiliser with pour garantir la fermeture de la connexion même en cas d'erreur.

```
import sqlite3

# "with" gère l'ouverture et la fermeture automatiquement
with sqlite3.connect(":memory:") as conn:
    cursor = conn.cursor()

    cursor.execute("""
        CREATE TABLE client (
            id    INTEGER PRIMARY KEY AUTOINCREMENT,
            nom   TEXT NOT NULL,
            email TEXT NOT NULL
        )
    """)

    # Insertion de plusieurs enregistrements d'un coup
    clients = [
        ("Bechir", "bechir@mail.com"),
        ("Sarrah", "sarrah@mail.com"),
        ("Karim", "karim@mail.com"),
    ]
    cursor.executemany(
        "INSERT INTO client (nom, email) VALUES (?, ?)",
        clients
    )

    conn.commit()

    # Lecture avec filtre
    cursor.execute("SELECT nom, email FROM client WHERE nom LIKE 'B%'")
    print("Clients dont le nom commence par B :")
    for row in cursor.fetchall():
        print(f"    {row[0]} → {row[1]}")

# Connexion fermée automatiquement ici
print("Connexion fermée □")

Clients dont le nom commence par B :
Bechir → bechir@mail.com
Connexion fermée □
```

- **with sqlite3.connect(...) as conn** : à la sortie du bloc, la connexion est fermée automatiquement —même si une exception survient

- `cursor.executemany(sql, liste)` : exécute la même requête pour chaque élément de la liste — bien plus efficace qu'une boucle exécute
- `SELECT ... WHERE nom LIKE 'B%'` : % est le joker SQL — B% = commence par B

## 4 □ Les 4 opérations CRUD

CRUD = **C**reate, **R**ead, **U**ppdate, **D**eleter — les 4 opérations fondamentales sur une base.

```
import sqlite3

conn = sqlite3.connect(":memory:")
cursor = conn.cursor()

cursor.execute("""
    CREATE TABLE produit (
        id    INTEGER PRIMARY KEY AUTOINCREMENT,
        nom   TEXT NOT NULL,
        prix  REAL NOT NULL
    )
""")

# — CREATE : insérer —————
cursor.execute("INSERT INTO produit (nom, prix) VALUES (?, ?)", ("Clavier", 99.9))
cursor.execute("INSERT INTO produit (nom, prix) VALUES (?, ?)", ("Souris", 49.9))
cursor.execute("INSERT INTO produit (nom, prix) VALUES (?, ?)", ("Écran", 299.0))
conn.commit()
print("INSERT □ — 3 produits ajoutés")

# — READ : lire —————
cursor.execute("SELECT id, nom, prix FROM produit ORDER BY prix")
print("\nSELECT — produits par prix croissant :")
for row in cursor.fetchall():
    print(f"  #{row[0]}  {row[1]:<10}  {row[2]:>8.2f} TND")

# — UPDATE : modifier —————
cursor.execute(
    "UPDATE produit SET prix = ? WHERE nom = ?",
    (89.9, "Clavier")
)
conn.commit()
print(f"\nUPDATE □ — {cursor.rowcount} ligne(s) modifiée(s)")
```

---

```

# — DELETE : supprimer
cursor.execute("DELETE FROM produit WHERE prix < ?", (50.0,))
conn.commit()
print(f"DELETE ? — {cursor.rowcount} ligne(s) supprimée(s)")

# Vérification finale
cursor.execute("SELECT nom, prix FROM produit")
print("\nÉtat final :")
for row in cursor.fetchall():
    print(f" {row[0]} → {row[1]:.2f} TND")

cursor.close()
conn.close()

INSERT ? — 3 produits ajoutés

SELECT — produits par prix croissant :
#2  Souris      49.90 TND
#1  Clavier     99.90 TND
#3  Écran      299.00 TND

UPDATE ? — 1 ligne(s) modifiée(s)
DELETE ? — 1 ligne(s) supprimée(s)

État final :
Clavier → 89.90 TND
Écran → 299.00 TND

```

- **cursor.rowcount** : nombre de lignes affectées par le dernier UPDATE ou DELETE —utile pour vérifier qu'on a bien touché quelque chose
  - **toujours commit() après INSERT/UPDATE/DELETE** —pas nécessaire pour SELECT
  - **ORDER BY prix** : tri côté base —plus efficace que trier la liste Python
- 

## 5 □ Les transactions —tout ou rien

Une **transaction** garantit qu'un groupe d'opérations s'exécute **entièrement ou pas du tout**. Indispensable pour les opérations liées (ex : virement bancaire).

```

import sqlite3

conn = sqlite3.connect(":memory:")
cursor = conn.cursor()

```

```

# Créer une table de comptes

```

```

cursor.execute("""
    CREATE TABLE compte (
        id      INTEGER PRIMARY KEY,
        nom     TEXT,
        solde   REAL
    )
""")
cursor.execute("INSERT INTO compte VALUES (1, 'Bechir', 1000)")
cursor.execute("INSERT INTO compte VALUES (2, 'Sarra', 500)")
conn.commit()

def virement(conn, id_from, id_to, montant):
    """Transfère montant du compte id_from vers id_to — atomique."""
    cursor = conn.cursor()
    try:
        # Opération 1 : débiter
        cursor.execute(
            "UPDATE compte SET solde = solde - ? WHERE id = ?",
            (montant, id_from)
        )
        # Simulation d'une erreur au milieu
        # raise Exception("Panne réseau !") # ← décommenter pour tester

        # Opération 2 : créditer
        cursor.execute(
            "UPDATE compte SET solde = solde + ? WHERE id = ?",
            (montant, id_to)
        )

        conn.commit() # Les DEUX opérations sont validées
        print(f" Virement {montant} TND : compte #{id_from} → compte #{id_to} ")

    except Exception as e:
        conn.rollback() # AUCUNE opération n'est appliquée
        print(f" Erreur : {e} → rollback effectué")

# Affichage des soldes
def afficher_soldes(conn):
    cur = conn.cursor()
    cur.execute("SELECT nom, solde FROM compte")
    for nom, solde in cur.fetchall():
        print(f" {nom:<10} : {solde:.2f} TND")

print("Avant virement :")
afficher_soldes(conn)

```

```
virement(conn, id_from=1, id_to=2, montant=200)
```

```
print("\nAprès virement :")
afficher_soldes(conn)
```

```
conn.close()
```

Avant virement :

Bechir : 1000.00 TND

Sarra : 500.00 TND

Virement 200 TND : compte #1 → compte #2 ☐

Après virement :

Bechir : 800.00 TND

Sarra : 700.00 TND

Sans transaction :

débiter Bechir ☐

☐ panne

créditer Sarra ? jamais

→ Bechir perd 200 TND

Sarra ne reçoit rien

Avec transaction :

BEGIN

débiter Bechir

☐ panne

ROLLBACK → rien n'a changé

→ état cohérent garanti ☐

- **conn.commit()** : valide **toutes** les opérations depuis le dernier commit —elles sont écrites sur disque
- **conn.rollback()** : annule **toutes** les opérations non committées —la base revient à son état précédent
- **toujours dans un try/except** : si une opération lève une exception, le rollback() est dans le except

---

## 6 ☐ row\_factory —accéder aux colonnes par nom

Par défaut, chaque ligne retournée est un **tuple** —on accède aux colonnes par index (row[0], row[1]). C'est fragile. row\_factory permet d'accéder par **nom de colonne**.

```
import sqlite3
```

```
# — Sans row_factory : accès par index
```

```
conn = sqlite3.connect(":memory:")
```

```
cursor = conn.cursor()
```

```
cursor.execute("CREATE TABLE client (id INTEGER, nom TEXT, email TEXT)")
```

```
cursor.execute("INSERT INTO client VALUES (1, 'Bechir', 'b@mail.com')")
```

```
conn.commit()
```

```

cursor.execute("SELECT * FROM client")
row = cursor.fetchone()
print("Sans row_factory :")
print(f" row[0]={row[0]} row[1]={row[1]} row[2]={row[2]}")
# → row[1] : qu'est-ce que c'est ? Pas évident à lire
conn.close()

# — Avec row_factory : accès par nom —————
conn = sqlite3.connect(":memory:")
conn.row_factory = sqlite3.Row # ← une ligne suffit
cursor = conn.cursor()
cursor.execute("CREATE TABLE client (id INTEGER, nom TEXT, email TEXT)")
cursor.execute("INSERT INTO client VALUES (1, 'Bechir', 'b@mail.com')")
conn.commit()

cursor.execute("SELECT * FROM client")
row = cursor.fetchone()
print("\nAvec row_factory :")
print(f" id={row['id']} nom={row['nom']} email={row['email']}")
# → clair, résistant aux changements de schéma

# Conversion en dict classique si besoin
print(f" dict : {dict(row)}")

conn.close()

Sans row_factory :
row[0]=1 row[1]=Bechir row[2]=b@mail.com

Avec row_factory :
id=1 nom=Bechir email=b@mail.com
dict : {'id': 1, 'nom': 'Bechir', 'email': 'b@mail.com'}

• conn.row_factory = sqlite3.Row : à placer juste après connect()
  —s'applique à tous les curseurs de cette connexion
• résistant aux changements : si on ajoute une colonne au milieu,
  row['nom'] continue de fonctionner —row[1] pointerait vers la
  mauvaise colonne
• dict(row) : convertit facilement en dict Python standard —pratique pour
  sérialiser en JSON

```

## 7 □ fetchone, fetchall, fetchmany

Trois façons de récupérer les résultats —chacune adaptée à un cas.



```

import sqlite3

conn = sqlite3.connect(":memory:")
conn.row_factory = sqlite3.Row
cursor = conn.cursor()

cursor.execute("CREATE TABLE log (id INTEGER, message TEXT, niveau TEXT)")
cursor.executemany(
    "INSERT INTO log VALUES (?, ?, ?)",
    [
        (1, "Démarrage serveur", "INFO"),
        (2, "Connexion DB OK", "INFO"),
        (3, "Requête lente 2.3s", "WARN"),
        (4, "Timeout connexion", "ERROR"),
        (5, "Service redémarré", "INFO"),
    ]
)
conn.commit()

```

---

```

# — fetchone() : une seule ligne
cursor.execute("SELECT * FROM log WHERE niveau = 'ERROR'")
premiere_erreur = cursor.fetchone()
print(f"Première erreur : {premiere_erreur['message']}")

```

---

```

# — fetchall() : toutes les lignes d'un coup
cursor.execute("SELECT * FROM log ORDER BY id")
tous = cursor.fetchall()
print(f"\nTotal : {len(tous)} entrées")

```

---

```

# — fetchmany(n) : n lignes à la fois — pour gros volumes
cursor.execute("SELECT * FROM log")
print("\nLecture par lots de 2 :)")
while True:
    lot = cursor.fetchmany(2)    # Lit 2 lignes à la fois
    if not lot:
        break
    for row in lot:
        print(f" [{row['niveau']}] {row['message']}")

```

```

cursor.close()
conn.close()

```

Première erreur : Timeout connexion

Total : 5 entrées

```
Lecture par lots de 2 :
[INFO] Démarrage serveur
[INFO] Connexion DB OK
[WARN] Requête lente 2.3s
[ERROR] Timeout connexion
[INFO] Service redémarré
```

| Méthode                   | Retourne                   | Quand l'utiliser                         |
|---------------------------|----------------------------|------------------------------------------|
| <code>fetchone()</code>   | 1 ligne ou None            | On cherche un enregistrement précis      |
| <code>fetchall()</code>   | Liste de toutes les lignes | Volume raisonnable (< quelques milliers) |
| <code>fetchmany(n)</code> | Liste de n lignes          | Gros volumes —traitement par lots        |

- **`fetchall()` charge tout en mémoire** —sur des millions de lignes, utiliser `fetchmany()` ou itérer sur le curseur directement
- **`fetchone()` retourne None** si aucun résultat —toujours vérifier avant d'accéder aux champs

## 8 □ Connexion à MySQL

MySQL nécessite un serveur installé séparément. La bibliothèque `mysql-connector-python` fournit le même pattern que SQLite, avec `%s` comme placeholder.

```
# pip install mysql-connector-python
```

```
import mysql.connector
from mysql.connector import Error

try:
    conn = mysql.connector.connect(
        host      = "localhost",
        user      = "root",
        password  = "monmotdepasse",
        database  = "boutique"
    )

    if conn.is_connected():
        print("□ Connexion MySQL réussie")

        cursor = conn.cursor()
```

```

cursor.execute("""
    CREATE TABLE IF NOT EXISTS produit (
        id      INT AUTO_INCREMENT PRIMARY KEY,
        nom     VARCHAR(100),
        prix    DECIMAL(10, 2)
    )
""")

# MySQL utilise %s (pas ?) comme placeholder
cursor.execute(
    "INSERT INTO produit (nom, prix) VALUES (%s, %s)",
    ("Clavier", 99.90)
)
conn.commit()

cursor.execute("SELECT * FROM produit")
for row in cursor.fetchall():
    print(row)

except Error as e:
    print(f"❌ Erreur MySQL : {e}")

finally:
    if 'conn' in locals() and conn.is_connected():
        cursor.close()
        conn.close()
        print("Connexion fermée")

```

#### Différences SQLite vs MySQL :

|                         | SQLite              | MySQL                 |
|-------------------------|---------------------|-----------------------|
| <b>Placeholder</b>      | ?                   | %s                    |
| <b>Auto-increment</b>   | AUTOINCREMENT       | AUTO_INCREMENT        |
| <b>Gestion erreurs</b>  | Exception générique | mysql.connector.Error |
| <b>Vérif. connexion</b> | Non nécessaire      | conn.is_connected()   |
| <b>Serveur requis</b>   | Non                 | Oui                   |

- **conn.is\_connected()** : une connexion MySQL peut être perdue (time-out réseau, redémarrage serveur) —toujours vérifier avant utilisation
- **finally** : garantit la fermeture de la connexion même si une exception survient dans try

## 9 □ Connexion à PostgreSQL

PostgreSQL est la base de référence pour les systèmes critiques. La bibliothèque psycopg2 utilise le même %s que MySQL.

```
# pip install psycopg2-binary
```

```
import psycopg2
from psycopg2 import OperationalError

try:
    conn = psycopg2.connect(
        host      = "localhost",
        database  = "boutique",
        user      = "postgres",
        password  = "monmotdepasse",
        port      = "5432"
    )
    print("□ Connexion PostgreSQL réussie")

    cursor = conn.cursor()

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS client (
            id      SERIAL PRIMARY KEY,  -- SERIAL = auto-increment PostgreSQL
            nom     VARCHAR(100),
            email   VARCHAR(100)
        )
    """)

    # PostgreSQL utilise aussi %s comme placeholder
    cursor.execute(
        "INSERT INTO client (nom, email) VALUES (%s, %s)",
        ("Bechir", "bechir@mail.com")
    )
    conn.commit()

    cursor.execute("SELECT * FROM client")
    for row in cursor.fetchall():
        print(row)

except OperationalError as e:
    print(f"□ Erreur PostgreSQL : {e}")

finally:
    if 'conn' in locals():
```

```

        cursor.close()
        conn.close()
        print("Connexion fermée")

```

**Les 3 bases côte à côte :**

|                       | SQLite               | MySQL                 | PostgreSQL                |
|-----------------------|----------------------|-----------------------|---------------------------|
| <b>Module</b>         | sqlite3<br>(intégré) | mysql.connector       | psycopg2                  |
| <b>Placeholder</b>    | ?                    | %s                    | %s                        |
| <b>Auto-increment</b> | AUTOINCREMENT        | AUTO_INCREMENT        | SERIAL                    |
| <b>Connexion</b>      | Fichier local        | host/user/password    | host/user/password/port   |
| <b>Erreur</b>         | Exception            | mysql.connector.Error | psycopg2.OperationalError |
| <b>Usage</b>          | Tests / local        | Web /<br>e-commerce   | Finance / critique        |

## □ Architecture DAO —séparer SQL et logique métier

En production, on ne mélange pas le SQL avec la logique de l'application. Le pattern **DAO** (Data Access Object) isole tout accès à la base dans une couche dédiée.

```

import sqlite3
from contextlib import contextmanager

```

```

# — Couche connexion
def get_connection():
    conn = sqlite3.connect(":memory:")
    conn.row_factory = sqlite3.Row
    return conn

```

```

# — BaseDAO : gestion transactionnelle réutilisable
class BaseDAO:

    @contextmanager
    def _session(self):
        """Ouvre une connexion, commit si succès, rollback si erreur."""
        conn = get_connection()
        try:
            yield conn
            conn.commit()
        except Exception:

```

```

        conn.rollback()
        raise
    finally:
        conn.close()

# — ClientDAO : tout le SQL lié aux clients —————
class ClientDAO(BaseDAO):

    def creer_table(self):
        with self._session() as conn:
            conn.execute("""
                CREATE TABLE IF NOT EXISTS client (
                    id      INTEGER PRIMARY KEY AUTOINCREMENT,
                    nom      TEXT NOT NULL,
                    email     TEXT NOT NULL
                )
            """)

    def inserer(self, nom, email):
        with self._session() as conn:
            conn.execute(
                "INSERT INTO client (nom, email) VALUES (?, ?)",
                (nom, email)
            )

    def lister(self):
        with self._session() as conn:
            cur = conn.execute("SELECT * FROM client ORDER BY nom")
            return cur.fetchall()

# — ClientService : logique métier, zéro SQL —————
class ClientService:

    def __init__(self):
        self.dao = ClientDAO()
        self.dao.creer_table()

    def ajouter(self, nom, email):
        if "@" not in email:
            raise ValueError(f"Email invalide : {email!r}")
        self.dao.inserer(nom, email)
        print(f" 📧 Client ajouté : {nom}")

    def lister(self):
        return self.dao.lister()

```

---

```
# — Programme principal
service = ClientService()

service.ajouter("Bechir", "bechir@mail.com")
service.ajouter("Sarra", "sarra@mail.com")

try:
    service.ajouter("Karim", "email-invalid") # ← erreur métier
except ValueError as e:
    print(f"❗ {e}")

print("\nListe des clients :")
for client in service.lister():
    print(f"#{client['id']} {client['nom']} — {client['email']}")

❑ Client ajouté : Bechir
❑ Client ajouté : Sarra
❑ Email invalide : 'email-invalid'
```

```
Liste des clients :
#1 Bechir — bechir@mail.com
#2 Sarra — sarra@mail.com
```

Architecture en couches :

```
main.py          → appelle ClientService (logique métier)
ClientService    → valide les données, appelle ClientDAO
ClientDAO        → exécute le SQL, hérite de BaseDAO
BaseDAO          → gère connexion, commit, rollback
get_connection() → fournit la connexion (SQLite/MySQL/Pg)
```

Avantage : changer de base = modifier uniquement `get_connection()`

- **@contextmanager** : transforme `_session` en gestionnaire de contexte — `yield conn` rend la connexion disponible dans le `with`
  - **Service sans SQL** : la validation métier ("`@`" not in `email`) est dans le service, pas dans le DAO — chaque couche a une responsabilité claire
  - **BaseDAO réutilisable** : `ProduitDAO`, `CommandeDAO` hériteront de `BaseDAO` et bénéficieront de la même gestion transactionnelle
- 

## 1 ❑1 ❑ Injection SQL — le danger à ne jamais ignorer

L'injection SQL est la vulnérabilité la plus répandue en développement web. Elle survient quand on **concatène** des données utilisateur dans une requête.

```

import sqlite3

conn = sqlite3.connect(":memory:")
cursor = conn.cursor()
cursor.execute("CREATE TABLE client (id INTEGER, nom TEXT, mot_de_passe TEXT)")
cursor.execute("INSERT INTO client VALUES (1, 'admin', 'secret123')")
conn.commit()

# — DANGER : concaténation directe —————
def login_dangereux(nom):
    requete = f"SELECT * FROM client WHERE nom = '{nom}'"
    print(f" Requête : {requete}")
    cursor.execute(requete)
    return cursor.fetchone()

# Utilisation normale
print("Connexion normale :")
print(" ", login_dangereux("admin"))

# Attaque par injection SQL
print("\nAttaque injection :")
print(" ", login_dangereux("' OR 1=1 --"))
# → La requête devient : WHERE nom = '' OR 1=1 --'
# → 1=1 est toujours vrai : retourne TOUS les utilisateurs !

# — SÉCURISÉ : paramètres —————
def login_securise(nom):
    cursor.execute("SELECT * FROM client WHERE nom = ?", (nom,))
    return cursor.fetchone()

print("\nAvec paramètres :")
print(" Admin :", login_securise("admin"))
print(" Attaque :", login_securise("' OR 1=1 --")) # None — attaque bloquée ☒

cursor.close()
conn.close()

Connexion normale :
Requête : SELECT * FROM client WHERE nom = 'admin'
(1, 'admin', 'secret123')

Attaque injection :
Requête : SELECT * FROM client WHERE nom = '' OR 1=1 --'
(1, 'admin', 'secret123') ← données volées !

Avec paramètres :

```



Admin : (1, 'admin', 'secret123')  
Attaque : None ← attaque bloquée

- `f"... {nom} ..."` dans une requête SQL : **jamais** —même si nom vient d'un formulaire "de confiance"
  - `cursor.execute(sql, (valeur,))` : Python échappe automatiquement la valeur —les caractères `'`, `"`, `--` ne peuvent pas "sortir" du paramètre
  - **règle absolue** : toute valeur externe (formulaire, URL, fichier, API) passe par les placeholders `?` (SQLite) ou `%s` (MySQL/PostgreSQL)
- 

## 1 2 Bonnes pratiques —résumé opérationnel

```
import sqlite3
```

```
# 1. Toujours utiliser "with" pour la connexion
```

```
with sqlite3.connect("boutique.db") as conn:
```

```
    pass # connexion fermée automatiquement
```

```
# 2. Toujours utiliser des paramètres — jamais de concaténation
```

```
# f"INSERT INTO t VALUES ('{valeur}')" 
```

```
# cursor.execute("INSERT INTO t VALUES (?)", (valeur,))
```

```
# 3. row_factory pour accès par nom de colonne
```

```
conn = sqlite3.connect(":memory:")
```

```
conn.row_factory = sqlite3.Row # ← juste après connect()
```

```
# 4. commit() après chaque modification
```

```
conn.commit() # après INSERT, UPDATE, DELETE
```

```
# 5. rollback() dans le except pour les transactions
```

```
try:
```

```
    pass # opérations
```

```
    conn.commit()
```

```
except Exception:
```

```
    conn.rollback()
```

```
    raise
```

```
# 6. fetchmany() pour les gros volumes
```

```
cursor = conn.cursor()
```

```
cursor.execute("SELECT * FROM grosse_table")
```

```
while lot := cursor.fetchmany(1000):
```

```
    for row in lot:
```

```
        pass # traiter par lots de 1000
```

---

### Guide de décision :

Quel moteur choisir ?

- Développement local, tests, app mobile
  - SQLite (aucune installation, module intégré)
- Application web, e-commerce, API
  - MySQL (rapide, bien supporté, hébergements faciles)
- Système critique, finance, données sensibles
  - PostgreSQL (transactions robustes, typage strict)

Règles universelles (valables pour les 3 moteurs) :

|                         |                           |
|-------------------------|---------------------------|
| with conn:              | fermeture automatique     |
| cursor.execute(sql, ()) | paramètres sécurisés      |
| conn.commit()           | valider les modifications |
| conn.rollback()         | annuler en cas d'erreur   |
| row_factory             | accès par nom de colonne  |
| f"... {variable} ..."   | injection SQL interdite   |

---

### □ Récapitulatif —les 6 points clés

**Connexion + curseur** : deux objets toujours nécessaires · **commit()** : obligatoire après toute modification · **rollback()** : annule tout en cas d'erreur · **paramètres ? ou %s** : seule façon sécurisée d'injecter des valeurs · **row\_factory** : accès par nom de colonne · **DAO** : isoler le SQL dans une couche dédiée

## 20 Questions intelligentes –Connexions bases de données

SQLite · MySQL · PostgreSQL

Format : Énoncé + Solution complète démonstrative

---

## 1) Pourquoi commit() est-il obligatoire après INSERT ?

### Énoncé

Pourquoi les données semblent insérées mais disparaissent après fermeture de connexion ?

### Solution

```
"""
```

*Explication :*

*Dans la plupart des moteurs relationnels, les modifications sont placées dans une transaction. Sans commit(), elles ne sont pas persistées.*

*SQLite peut parfois donner l'illusion que cela fonctionne, mais en réalité la transaction n'est jamais committée.*

```
import sqlite3
```

```
conn = sqlite3.connect("test_commit.db")
cursor = conn.cursor()
```

```
cursor.execute("CREATE TABLE IF NOT EXISTS test (id INTEGER PRIMARY KEY, nom TEXT)")
cursor.execute("INSERT INTO test (nom) VALUES (?)", ("Alice",))
```

```
# conn.commit() volontairement omis
```

```
cursor.close()
conn.close()
```

```
# Nouvelle connexion pour vérifier
```

```
conn = sqlite3.connect("test_commit.db")
cursor = conn.cursor()
cursor.execute("SELECT * FROM test")
print("Contenu après fermeture sans commit:", cursor.fetchall())
```

```
cursor.close()
conn.close()
```

Observation : les données ne sont pas persistées.

## 2) Pourquoi ne jamais concaténer une variable dans une requête SQL ?

### Énoncé

Quel est le risque si on écrit :

```
SELECT * FROM client WHERE nom = "" + nom + ""?
```

### Solution

```
"""
```

*Explication :*

*Concaténer une variable ouvre la porte aux injections SQL.*

*Les paramètres sécurisés empêchent l'exécution de code malveillant.*

```
"""
```

```
import sqlite3
```

```
conn = sqlite3.connect(":memory:")
cursor = conn.cursor()
```

```
cursor.execute("CREATE TABLE client (id INTEGER PRIMARY KEY, nom TEXT)")
cursor.execute("INSERT INTO client (nom) VALUES (?)", ("Alice",))
conn.commit()
```

```
# Simulation injection
```

```
nom = "' OR 1=1 --"
```

```
requete_dangereuse = "SELECT * FROM client WHERE nom = '" + nom + "'"
print("Requête dangereuse:", requete_dangereuse)
```

```
cursor.execute(requete_dangereuse)
print("Résultat injection:", cursor.fetchall())
```

```
# Version sécurisée
```

```
cursor.execute("SELECT * FROM client WHERE nom = ?", (nom,))
print("Résultat sécurisé:", cursor.fetchall())
```

```
cursor.close()
conn.close()
```

---

### 3) Pourquoi faut-il gérer rollback() ?

#### Énoncé

Que se passe-t-il si une erreur survient entre deux UPDATE ?

#### Solution

```
"""
```

*Explication :*

*Sans rollback(), la base peut rester dans un état incohérent.*

*Une transaction doit être atomique.*

```
"""
```

```
import sqlite3
```

```
conn = sqlite3.connect(":memory:")
cursor = conn.cursor()
```

```
cursor.execute("CREATE TABLE compte (id INTEGER PRIMARY KEY, solde INTEGER)")
cursor.execute("INSERT INTO compte VALUES (1, 1000)")
cursor.execute("INSERT INTO compte VALUES (2, 500)")
conn.commit()
```

```
try:
```

```
    cursor.execute("UPDATE compte SET solde = solde - 100 WHERE id = 1")
    raise Exception("Erreur simulée")
    cursor.execute("UPDATE compte SET solde = solde + 100 WHERE id = 2")
    conn.commit()
```

```
except Exception:
```

```
    conn.rollback()
    print("Rollback effectué")
```

```
cursor.execute("SELECT * FROM compte")
print("État final:", cursor.fetchall())
```

```
cursor.close()
conn.close()
```

---

## 4) Différence entre fetchone() et fetchall() ?

### Énoncé

Quand utiliser l'un ou l'autre ?

### Solution

```
"""
Explication :
fetchone() récupère une seule ligne.
fetchall() charge tout en mémoire.
Pour gros volumes, fetchone() est plus sûr.
"""

import sqlite3

conn = sqlite3.connect(":memory:")
cursor = conn.cursor()

cursor.execute("CREATE TABLE test (id INTEGER)")
cursor.executemany("INSERT INTO test VALUES (?)", [(1,), (2,), (3,)])
conn.commit()

cursor.execute("SELECT * FROM test")
print("fetchone:", cursor.fetchone())

cursor.execute("SELECT * FROM test")
print("fetchall:", cursor.fetchall())

cursor.close()
conn.close()
```

---

## 5) Pourquoi utiliser row\_factory avec SQLite ?

### Énoncé

Comment accéder aux colonnes par nom plutôt que par index ?

### Solution

```
"""
Explication :
```

*row\_factory = sqlite3.Row permet d'accéder aux colonnes par clé.  
Plus lisible et robuste.*

"""

```
import sqlite3

conn = sqlite3.connect(":memory:")
conn.row_factory = sqlite3.Row
cursor = conn.cursor()

cursor.execute("CREATE TABLE client (id INTEGER, nom TEXT)")
cursor.execute("INSERT INTO client VALUES (?, ?)", (1, "Alice"))
conn.commit()

cursor.execute("SELECT * FROM client")
row = cursor.fetchone()

print("Accès par index:", row[1])
print("Accès par nom:", row["nom"])

cursor.close()
conn.close()
```

---

## 6) Que se passe-t-il si on ferme la connexion avant fetchall() ?

### Énoncé

Pourquoi cela provoque une erreur ?

### Solution

"""

*Explication :*  
*Le curseur dépend de la connexion.*  
*Fermer la connexion invalide le curseur.*

"""

```
import sqlite3

conn = sqlite3.connect(":memory:")
cursor = conn.cursor()
```

```

cursor.execute("CREATE TABLE test (id INTEGER)")
cursor.execute("INSERT INTO test VALUES (1)")
conn.commit()

cursor.execute("SELECT * FROM test")
conn.close()

try:
    print(cursor.fetchall())
except Exception as e:
    print("Erreur:", e)

```

---

## 7) Pourquoi tester conn.is\_connected() en MySQL ?

### Énoncé

Que se passe-t-il si la connexion est perdue ?

### Solution

```

"""
Explication :
Une connexion peut être interrompue (timeout, réseau).
Toujours vérifier avant utilisation.
"""

import mysql.connector

conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="password",
    database="testdb"
)

print("Connexion active:", conn.is_connected())

conn.close()

```

---



## 8) Pourquoi utiliser un contextmanager ?

### Énoncé

Comment garantir fermeture automatique ?

### Solution

```
"""
Explication :
with garantit fermeture même en cas d'exception.
Évite fuite de connexions.
"""

import sqlite3

with sqlite3.connect(":memory:") as conn:
    cursor = conn.cursor()
    cursor.execute("CREATE TABLE test (id INTEGER)")
    cursor.execute("INSERT INTO test VALUES (1)")
    conn.commit()

print("Connexion fermée automatiquement")
```

---

## 9) Pourquoi éviter SELECT \* en production ?

### Énoncé

Quel est le problème ?

### Solution

```
"""
Explication :
SELECT * :
- Récupère colonnes inutiles
- Rend code fragile si schéma change
Bonne pratique : sélectionner explicitement.
"""

import sqlite3

conn = sqlite3.connect(":memory:")
```

```

cursor = conn.cursor()

cursor.execute("CREATE TABLE test (id INTEGER, nom TEXT)")
cursor.execute("INSERT INTO test VALUES (1, 'Alice')")
conn.commit()

cursor.execute("SELECT id FROM test")
print(cursor.fetchall())

cursor.close()
conn.close()

```

---

## 10) Pourquoi utiliser des index ?

### Énoncé

Quel impact sur performance ?

### Solution

```

"""
Explication :
Un index accélère les recherches sur colonnes filtrées.
Sans index, scan complet.
"""

import sqlite3

conn = sqlite3.connect(":memory:")
cursor = conn.cursor()

cursor.execute("CREATE TABLE test (id INTEGER, nom TEXT)")
cursor.execute("CREATE INDEX idx_nom ON test(nom)")
print("Index créé")

cursor.close()
conn.close()

```

---

## 11) Différence autocommit MySQL vs PostgreSQL ?

### Énoncé

Pourquoi comportement différent ?

### Solution

```
"""
```

*Explication :*

*MySQL peut être configuré en autocommit.*

*PostgreSQL démarre transaction implicitement.*

*Toujours maîtriser ce comportement.*

```
"""
```

```
import psycopg2
```

```
conn = psycopg2.connect(  
    host="localhost",  
    database="testdb",  
    user="postgres",  
    password="password"  
)
```

```
print("Autocommit par défaut:", conn.autocommit)  
conn.close()
```